DuoPay: Low-Level Design (LLD)

Document	Low-Level Design, v1.0
Implements	HLD
Companion specs	API spec (every endpoint) · Database (DDL, indexes, retention)

This LLD describes the prototype as built: Node.js 22, no third-party packages, SQLite through `node:sqlite`, cryptography through `node:crypto` and WebCrypto. Every external party (telecom, PSP, NPCI, issuer banks, NPCI Common Library) is a **simulator behind the same interface a real adapter would implement**.

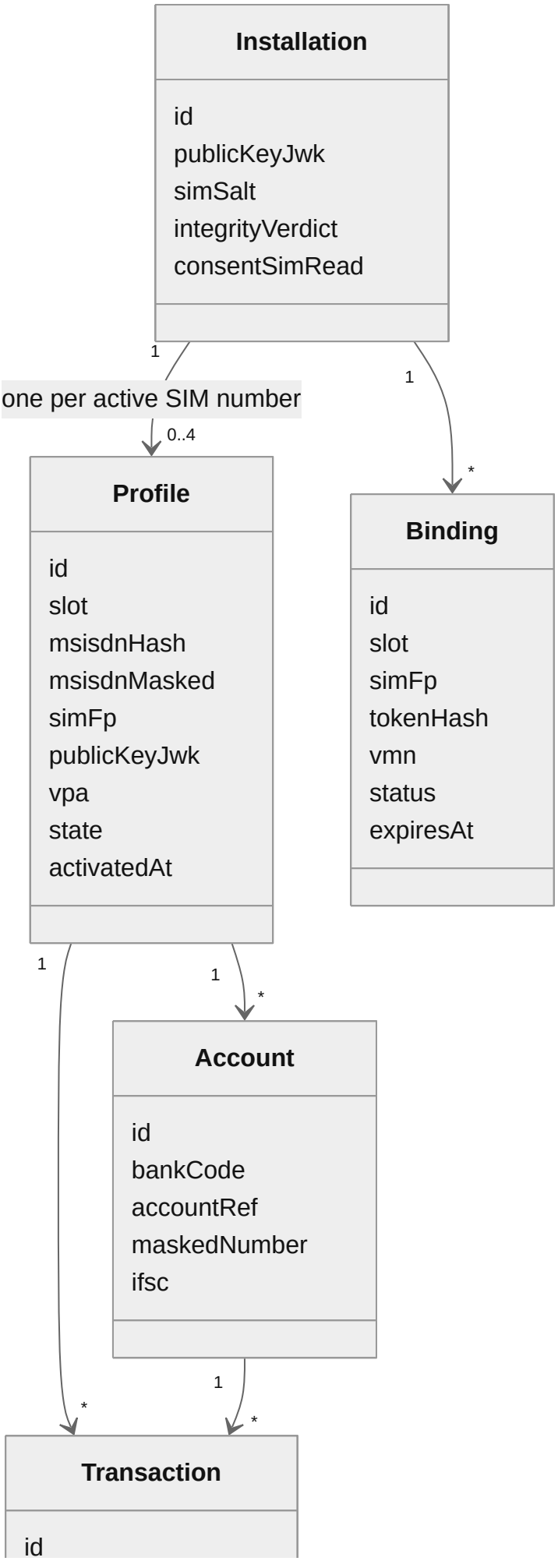
---

1. Code layout

```
src/
  config.js                env + defaults (limits, timers, feature flags)
  server.js                starts the HTTP server
  app.js                   composition root: DB, simulators, adapter, route table, static files,
error mapping
  http/
    auth.js                request-signature verification, nonce replay, SIM-state header
    errors.js              AppError + error codes → HTTP status
  db/
    schema.sql             DDL (see database.md)
    index.js               open, migrate, transaction helper
  lib/
    crypto.js              ids, tokens, AES-GCM for MSISDN, HMAC, hashing
    time.js                IST day boundaries, clock injection for tests
  modules/
    installations.js       register install, consent
    bindings.js            SMS device-binding protocol (per SIM)
    profiles.js            profile state machine (only writer of profiles.state)
    simIntegrity.js        compare reported SIM state with bound fingerprints
    accounts.js            account discovery (ReqListAccount) + linking
    limits.js              per-account daily limits, cooling-off, NPCI API quotas
    risk.js                deterministic risk rules
    advisor.js             eligibility + scoring + Claude re-ranking
    payments.js            suggest, pay, status check, reconciliation
    audit.js               hash-chained audit ledger
  ai/
    claude.js              Messages API client (forced tool call)
  adapters/
    psp.js                 interface used by modules → simulator today, real PSP later
  simulators/
    world.js               fictional SIM cards, numbers, banks, accounts, PINs, keys
    telecom.js             SMS from a SIM to a VMN (adds originating MSISDN)
    pspSwitch.js           PSP + NPCI + issuer banks (ReqListAccount, ReqValAdd, ReqPay,
ReqChkTxn, ListKeys)
public/
  index.html, app.js       phone simulator (2 SIM slots) + DuoPay app UI + ops console
  cl.js                    NPCI Common Library simulator (PIN → RSA-OAEP encrypted block)
test/                      node:test unit + scenario tests
scripts/security-lint.js   banned-pattern lint (also a Claude Code PostToolUse hook)
```

Dependency rule: `http → modules → adapters → simulators`. Modules never import simulators directly. `profiles.js` is the only module that writes `profiles.state`.

2. Domain model


Installation
id
publicKeyJwk
simSalt
integrityVerdict
consentSimRead
1
one per active SIM number
0..4
1
*
Profile
id
slot
msisdnHash
msisdnMasked
simFp
publicKeyJwk
vpa
state
activatedAt
Binding
id
slot
simFp
tokenHash
vmn
status
expiresAt
1
1
*
Account
id
bankCode
accountRef
maskedNumber
ifsc
1
*
*
Transaction
id

idempotencyKey
payeeVpa
amountPaise
status
rrn

3. Security primitives

3.1 Identifiers and tokens (`lib/crypto.js`)

Item	Generation
IDs	`prefix_` + 16 random bytes base64url (`ins_`, `prf_`, `bnd_`, `acc_`, `txn_`)
Binding token	40 characters from `[A-Za-z0-9!#$%*+=?@^_~]` using `crypto.randomInt`, so it is ≥35 characters with a mix of character types (HLD C1). **Only its SHA-256 is stored**
MSISDN at rest	`AES-256-GCM(DATA_KEY)` as `iv:tag:ct` (base64url) + `HMAC-SHA256(DATA_KEY, msisdn)` for unique lookups + masked form `98XXXXXX01`
SIM fingerprint (device side)	`SHA-256(simSalt + ':' + iccid)`, hex. The salt is issued per installation, so fingerprints can't be linked across installations
Audit hash	`HMAC-SHA256(DATA_KEY, prev_hash + '

3.2 Request signing (`http/auth.js`)

Each installation and each profile has its own ECDSA P-256 key pair. The private key never leaves the device: a non-extractable WebCrypto key stands in for the Android Keystore.

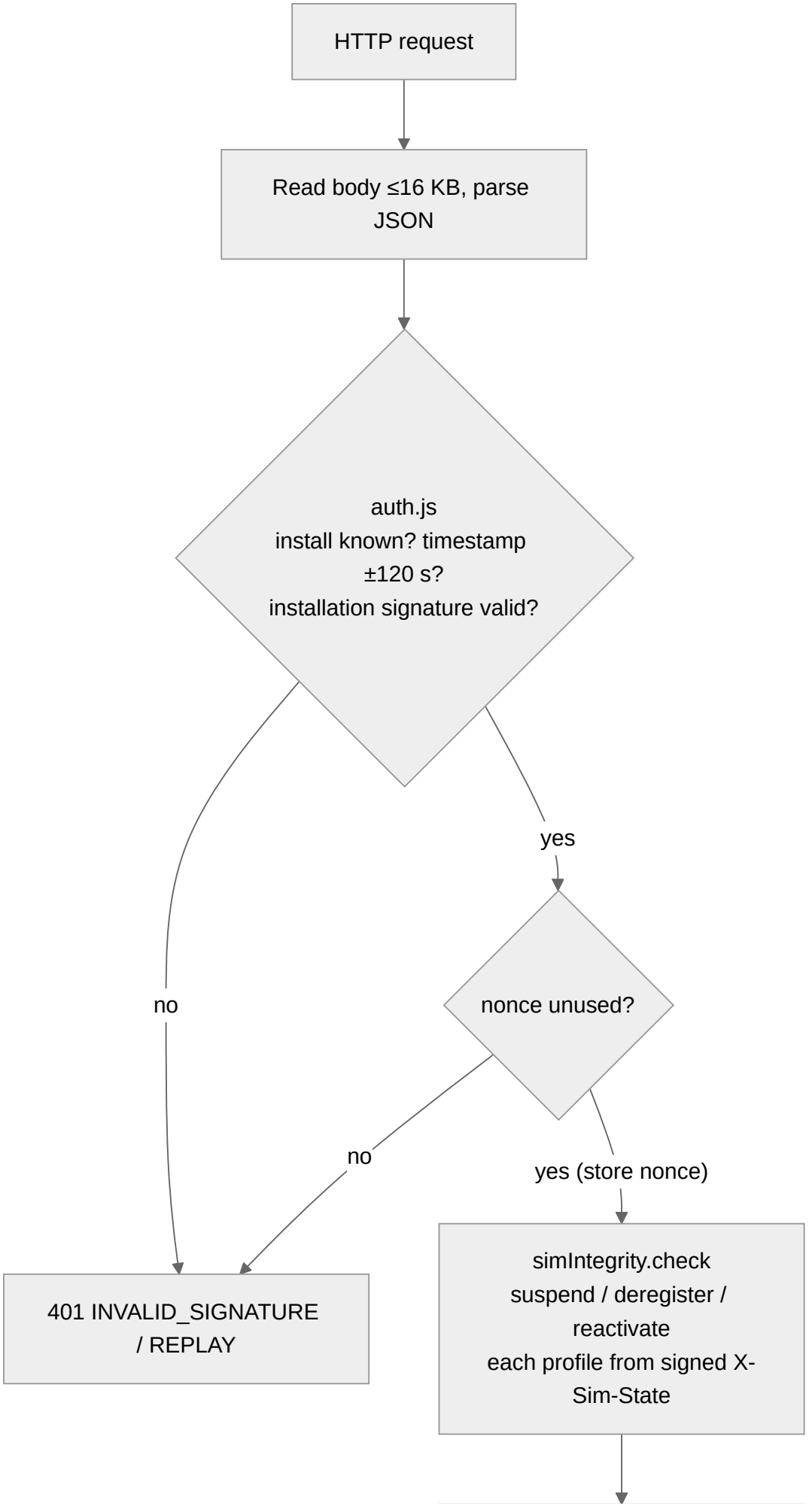
Canonical string signed by the device:

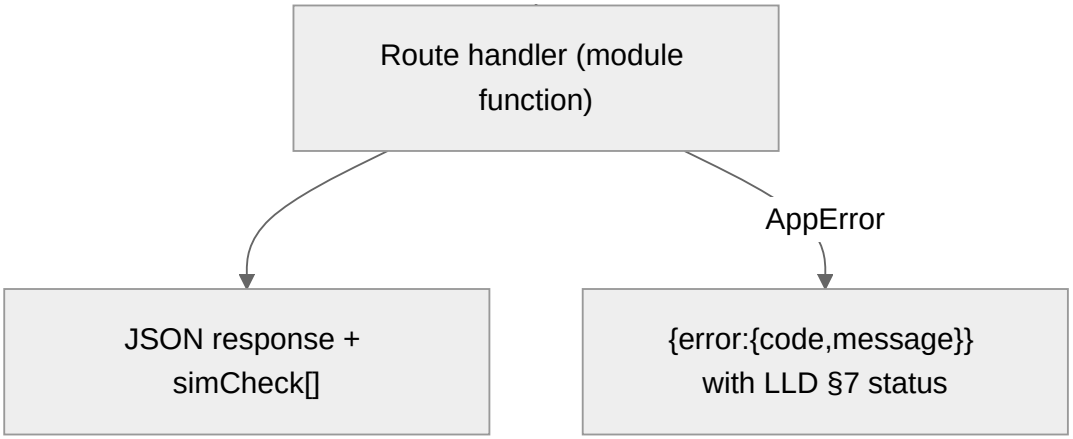
```
METHOD \n PATH \n X-Timestamp \n X-Nonce \n SHA256(body) \n SHA256(X-Sim-State)
```

Header	Meaning
`X-Install-Id`	Installation id
`X-Timestamp`	Unix ms. Rejected if more than ±120 s from server time
`X-Nonce`	16+ random bytes. Stored for 10 minutes; a repeat is rejected with `REPLAY`
`X-Sim-State`	base64url JSON `{airplane, slots:[{slot, state:'READY'
`X-Signature`	base64url ECDSA signature (IEEE-P1363) with the **installation** key
`X-Profile-Signature`	Same canonical string signed with the **profile** key. Required for payments and profile removal

Verification order: install exists → timestamp → installation signature → nonce unused → (if required) profile signature with **the key of the profile that owns the account being debited**. The nonce is recorded in its own transaction **before** any business logic runs, so a request that later fails can't be replayed either.

3.3 Request pipeline (every signed call)


HTTP request
Read body ≤16 KB, parse JSON
auth.js
install known? timestamp ±120 s?
installation signature valid?
no
yes
nonce unused?
no
yes (store nonce)
401 INVALID_SIGNATURE / REPLAY
simIntegrity.check
suspend / deregister / reactivate
each profile from signed X-Sim-State



> **Limitation (documented):** SIM state is reported by the device. In production it is backed by Play Integrity attestation and, where operators offer it, a telco SIM-swap API check at binding and before high-value payments.

---

4. Module designs

4.1 `bindings.js`: device binding per SIM

```
startBinding(installationId, {slot, profilePublicKeyJwk}, simState)
  1. simState.airplane == false                        else AIRPLANE_MODE
  2. slot in simState.slots with state READY           else SIM_ABSENT
  3. no live profile in this slot                      else SLOT_ALREADY_BOUND
  4. live profiles on installation < MAX_PROFILES (2)  else PROFILE_CAP
  5. attempts(installation, slot, 24h) < 3 AND attempts(installation, 24h) < 3 × MAX_PROFILES
else BINDING_LIMIT
     (never counted by the client-supplied fingerprint, which an attacker can rotate)
  5a. no other non-expired PENDING binding on this installation                    else
BINDING_IN_PROGRESS
  6. token = randomToken(40); vmn = random(VMN_POOL[10])
  7. INSERT bindings(status PENDING, token_hash, carrier = slot's MCC-MNC, expires_at = now + 45s)
  8. audit BINDING_STARTED
  return {bindingId, token, vmn, expiresAt}

onSmsReceived({vmn, text, originMsisdn})        ← called by the PSP adapter
  1. b = binding by sha256(text); none → ignore (log UNKNOWN_TOKEN)
  2. if b.received_msisdn_hash set AND ≠ hmac(originMsisdn) → FAILED TOKEN_MULTI_SOURCE (HLD C1);
a profile already created from the first SMS is deregistered
  3. if now > b.expires_at → EXPIRED
  4. if vmn ≠ b.vmn → FAILED VMN_MISMATCH
  4a. if MNP-operator(originMsisdn) ≠ b.carrier → FAILED SIM_NUMBER_MISMATCH   (ADR-010: token
relayed from a SIM in another phone)
  4b. if this number already has a live profile on THIS installation → FAILED
NUMBER_ALREADY_BOUND_ON_THIS_DEVICE
  4c. if ≥3 bindings received from this number in 24h (any installation) → FAILED BINDING_LIMIT
(survives re-installs)
  4d. re-check: slot still free and live profiles < MAX_PROFILES → else FAILED SLOT_ALREADY_BOUND
/ PROFILE_CAP
  4e. profile insert hits a constraint → FAILED PROFILE_CONFLICT (fail closed; token consumed)
  5. existing live profile for this msisdn on ANOTHER installation
       → profiles.deregister(old, 'REBOUND_ON_OTHER_DEVICE')  (same as normal UPI device change)
  6. profile = profiles.create({slot, msisdn, simFp, key, vpa}) in state BOUND
  7. b.status = VERIFIED, b.profile_id = profile.id; audit BINDING_VERIFIED

getBinding(id) → {status, profile?}   (expired PENDING is lazily marked EXPIRED)
```

VPA format: `dp<64 random bits in base36>.<slot>@duopay`. It is unique across **all** rows, so a VPA is never reissued, even after deregistration (a recycled VPA could route money to a stranger). It never contains the phone number.

4.2 `profiles.js`: state machine

From	Event	To	Side effects
—	`create`	BOUND	audit
BOUND	`accountsLinked(n ≥ 1)`	ACTIVE	`activated_at = now` (cooling-off starts)
ACTIVE	`simAbsent` / `airplane`	SUSPENDED	`suspended_at`, reason
ACTIVE / SUSPENDED	`simChanged` (different fp in slot)	DEREGISTERED	reason `SIM_CHANGED`
SUSPENDED	`simPresent` (same fp)	ACTIVE	`activated_at = now` (cooling-off restarts)
SUSPENDED	absent > `SUSPEND_TTL` (7 days)	DEREGISTERED	swept on next touch
any live	`remove` (user)	DEREGISTERED	requires profile signature
any live	`reboundElsewhere`	DEREGISTERED	

`transition(profile, event)` rejects anything not in the table with `INVALID_TRANSITION`. It is the only function that issues `UPDATE profiles SET state`.

4.3 `simIntegrity.js`

```
check(installationId, simState) → [{profileId, action}]
  for each live profile p of installation:
    s = simState.slots[p.slot]
    if simState.airplane            → action SUSPEND (AIRPLANE)
    elif !s or s.state == ABSENT    → action SUSPEND (SIM_ABSENT)
    elif s.fp != p.sim_fp           → action DEREGISTER (SIM_CHANGED)
    elif p.state == SUSPENDED       → action REACTIVATE
    else                            → OK
  apply actions via profiles.transition; audit each change
```

It runs as **middleware on every signed request**, so the latest SIM state is enforced before any business logic. It also runs on `POST /v1/sim-state`, the heartbeat the app sends on SIM-change broadcasts.

4.4 `accounts.js`

```
discover(installationId, profileId, bankCode)
  1. profile BOUND or ACTIVE; this request's signed SIM state shows the profile's slot READY with
the bound fingerprint and airplane off
     (SIM integrity doesn't suspend BOUND profiles, so discovery checks this itself)
  2. limits.consumeQuota(installationId, 'LIST_ACCOUNT')   ≤ 25/day per installation (shared by
all profiles)
  3. psp.listAccounts({msisdn: decrypt(profile.msisdn_enc), bankCode})   ← ReqListAccount
  4. upsert accounts (masked number, IFSC, type, opaque ref). Holder name and balance are NOT
stored
  5. if profile BOUND and ≥1 account → profiles.transition(ACTIVATE)
```

Discovery is allowed only for profiles in `BOUND` or `ACTIVE`, inside the onboarding UI (HLD C3). There is no background refresh.

4.5 `limits.js`

Limit	Default (config)	Scope	Code
Per transaction	₹1,00,000	transaction	`LIMIT_EXCEEDED`
Daily amount	₹1,00,000 (overridable per bank)	account, IST calendar day, SUCCESS + PENDING	`LIMIT_EXCEEDED`
Daily count	20	account	`LIMIT_EXCEEDED`
Cooling-off	₹5,000 total in the first 24h after `activated_at`	profile	`COOLING_OFF_LIMIT`
List account	25/day	installation	`QUOTA_EXCEEDED`
Status check	≤3 per txn within 2h, ≥90 s apart	transaction	`STATUS_CHECK_TOO_SOON` / `STATUS_CHECK_LIMIT`

`headroom(account)` returns `{amountLeft, countLeft, coolingLeft}`. It is used by both eligibility and the payment check. These figures are **as seen by DuoPay**: the issuer bank enforces its own limits across all apps, and its answer is final.

4.6 `risk.js`

```
evaluate({installationId, profile, account, payeeVpa, amountPaise}) → {decision, rules[]}
  R1 DEVICE_VELOCITY  > 5 payments in 5 min across ALL profiles on the installation → BLOCK
  R2 DEVICE_AMOUNT    > ₹2,00,000 in 24h across all profiles on the installation → BLOCK
  R3 NEW_PAYEE_HIGH   first payment to payeeVpa from this installation and amount > ₹20,000 →
CONFIRM
  R4 RECENT_REACTIVATION  profile activated < 1h ago and amount > ₹2,000 → CONFIRM
decision = BLOCK if any BLOCK, else CONFIRM if any CONFIRM (client must resend with confirm=true),
else ALLOW
```

R1 and R2 are the device-level controls ADR-004 relies on: several SIM numbers on one phone don't multiply the limits.

4.7 `advisor.js`: account suggestion

Step 1, eligibility (rules). For every account of the installation:

- the profile is `ACTIVE` (after SIM integrity ran);
- `amount ≤ headroom.amountLeft`, `countLeft > 0`, and if in cooling-off `amount ≤ coolingLeft`;
- the bank isn't flagged `DOWN` by the PSP.

Ineligible accounts are returned with a reason (`SIM_ABSENT`, `DAILY_LIMIT`, `COOLING_OFF`, `BANK_DOWN`) so the UI can show why they're greyed out.

Step 2, features for each eligible account (no PII, no balances):

```
{ ref: "A1", bank: "Bank C", payeeShare: 0.8, successRate7d: 0.97, headroomPct: 0.62,
lastUsedDaysAgo: 2 }
```

- `payeeShare`: the share of past payments to this payee made from this account.
- `successRate7d`: SUCCESS / (SUCCESS + FAILURE) for the bank over 7 days (prior 0.95 when there are fewer than 5 payments).

Step 3, deterministic score (always computed): `score = 0.45·payeeShare + 0.25·successRate7d + 0.20·headroomPct + 0.10·recency`, where `recency = 1/(1+lastUsedDaysAgo)`.

Step 4, Claude (optional), when `ANTHROPIC_API_KEY` is set and there are ≥2 eligible accounts:

- System prompt: rank the eligible accounts for this payment and give one reason of ≤ 90 characters, using only the given features.
- Forced tool `rank_accounts {ranking: string[], reason: string}`, temperature 0, 800 ms timeout.
- **Validation:** `ranking` must be a permutation of the eligible refs, and `reason` must contain no number absent from the features. On failure, timeout or error, use the deterministic order and a template reason (`source: "rules"`).

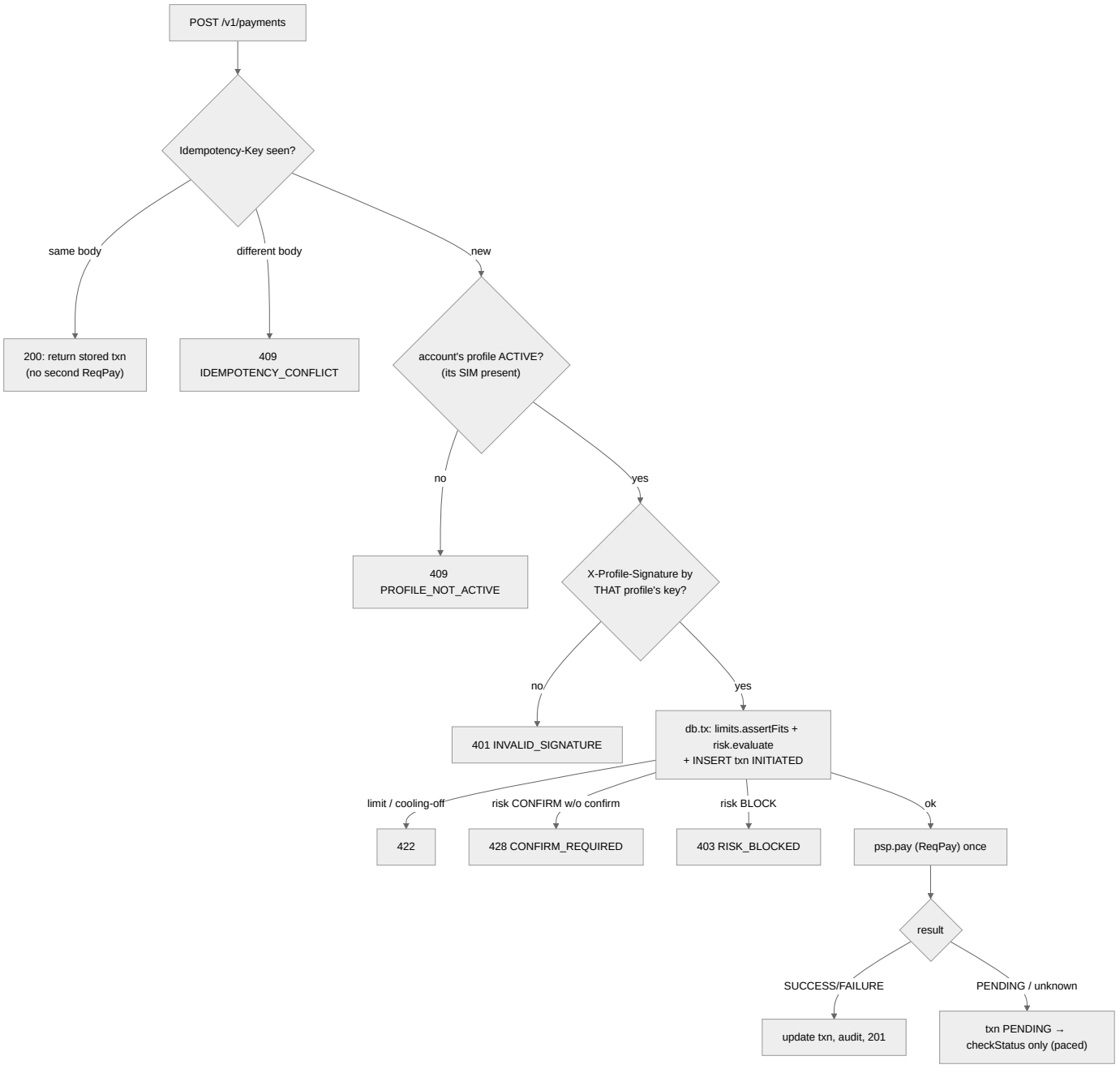
Response: `{eligible: [...ranked], ineligible: [...], source: "claude"|"rules", reason}`.

4.8 `payments.js`

```
suggest(installationId, {payeeVpa, amountPaise})
  psp.validateAddress(payeeVpa)  ← ReqValAdd → payee name or PAYEE_INVALID
  advisor.rank(...)

pay(installationId, {accountId, payeeVpa, amountPaise, credBlock, confirm, suggestedRank},
idempotencyKey, profileSig)
  1. existing = txn by (installationId, idempotencyKey)
       same request_hash → return existing (no second ReqPay)
       different hash   → IDEMPOTENCY_CONFLICT
  2. account → profile; profile.state == ACTIVE else PROFILE_NOT_ACTIVE
  3. verify X-Profile-Signature with profile.public_key_jwk else INVALID_SIGNATURE
  4. limits.check + risk.evaluate (CONFIRM without confirm=true → CONFIRM_REQUIRED)
  5. INSERT txn INITIATED; audit PAYMENT_INITIATED
  6. r = psp.pay({txnId, payerVpa: profile.vpa, accountRef, bankCode, payeeVpa, amount,
credBlock})  ← ReqPay
  7. UPDATE txn status = r.status (SUCCESS | FAILURE | PENDING), rrn, failure_code; audit

checkStatus(installationId, txnId)
  only for PENDING; enforce ≤3 checks in 2h and ≥90 s apart
  r = psp.checkTxn(txnId)  ← ReqChkTxn
  after the 3rd PENDING result → DEEMED_PENDING (user told: bank settles or reverses per NPCI TAT)
  NEVER re-sends ReqPay
```



The credential block is opaque to DuoPay. It is passed through to the PSP and is **never logged or stored**.

4.9 `audit.js`

`append(event, {installationId, profileId, data})` runs inside the caller's DB transaction: read the last hash, compute the new **HMAC-SHA256 (keyed with `DATA_KEY`)**, insert. The key means someone with DB access alone can't rewrite and re-chain the ledger. `verify()` walks the chain and reports the first broken `seq`. Payloads hold masked values only, enforced by an allow-list of keys per event type.

---

5. NPCI Common Library simulation (`public/cl.js`)

1. The app requests the issuer's public key (`GET /sim/npci/keys?bank=BNKC`, mirroring `ListKeys`).
2. The CL shows its own PIN pad, encrypts `{pin, amountPaise, payeeVpa, nonce, ts}` with RSA-OAEP-256 using the issuer key, and returns a base64 block.

3. The DuoPay app code never handles the PIN variable. The issuer simulator decrypts the block and checks the PIN, that amount and payee match the `ReqPay`, freshness (`ts` within 120 s) and that the nonce is unused. A captured block therefore can't be replayed or reused for a different amount.

6. PSP adapter interface (`adapters/psp.js`)

Method	UPI message it stands for	Returns
`listAccounts({msisdn, bankCode})`	ReqListAccount	`[{accountRef, maskedNumber, ifsc, type}]`
`validateAddress(vpa)`	ReqValAdd	`{valid, name}`
`pay({...})`	ReqPay	`{status, rrn?, code?}`
`checkTxn(txnId)`	ReqChkTxn	`{status, rrn?}`
`bankStatus()`	ListAccPvd (health)	`{BNKA: 'UP', ...}`
`onSms(handler)`	inbound binding SMS	—

7. Error codes

Code	HTTP	Meaning
`INVALID_SIGNATURE`	401	Missing or bad installation/profile signature
`REPLAY`	401	Nonce reused or timestamp out of window
`AIRPLANE_MODE`	409	Binding, linking or payment attempted in airplane mode
`BINDING_IN_PROGRESS`	409	Another SIM's verification is still pending on this device
`SIM_ABSENT`	409	Required SIM slot is empty or not ready
`SIM_CHANGED`	409	Different SIM in a bound slot; profile deregistered
`SLOT_ALREADY_BOUND`	409	Slot already has a live profile
`PROFILE_CAP`	409	Installation already has the maximum live profiles
`BINDING_LIMIT`	429	3 binding attempts in 24h for this SIM
`TOKEN_EXPIRED` / `TOKEN_MULTI_SOURCE` / `VMN_MISMATCH` / `SIM_NUMBER_MISMATCH` / `NUMBER_ALREADY_BOUND_ON_THIS_DEVICE`	—	Binding failure reasons, returned as `failReason` by `GET /v1/bindings/{id}`
`PROFILE_NOT_ACTIVE`	409	Payment from a profile that is not ACTIVE
`QUOTA_EXCEEDED`	429	NPCI per-app API quota used up
`LIMIT_EXCEEDED` / `COOLING_OFF_LIMIT`	422	Amount or count limits
`CONFIRM_REQUIRED`	428	A risk rule needs explicit confirmation
`RISK_BLOCKED`	403	A risk rule blocked the payment
`PAYEE_INVALID`	422	VPA failed validation
`BANK_DOWN`	503	Issuer unavailable

Code	HTTP	Meaning
`IDEMPOTENCY_CONFLICT`	409	Same key, different body
`STATUS_CHECK_TOO_SOON` / `STATUS_CHECK_LIMIT`	429	NPCI status-check pacing
`INVALID_TRANSITION`	409	Illegal profile state change
`VALIDATION` / `NOT_FOUND`	400 / 404	Bad input / unknown resource

8. Configuration (`config.js`, env overrides)

Key	Default	
`PORT`	3000	
`DB_PATH`	`./data/duopay.db`	`:memory:` in tests
`DATA_KEY`	generated per run with a warning	32-byte hex; required in production
`MAX_PROFILES`	2	per installation
`BINDING_TTL_SEC` / `BINDING_MAX_ATTEMPTS`	45 / 3	
`COOLING_OFF_PAISE` / `COOLING_OFF_HOURS`	500000 / 24	
`DAILY_LIMIT_PAISE` / `DAILY_COUNT` / `TXN_LIMIT_PAISE`	10000000 / 20 / 10000000	
`STATUS_CHECK_GAP_SEC` / `STATUS_CHECK_MAX`	90 / 3	demo may lower the gap
`SUSPEND_TTL_DAYS`	7	
`ANTHROPIC_API_KEY`, `CLAUDE_MODEL`, `ADVISOR_TIMEOUT_MS`	—, `claude-haiku-4-5`, 800	the advisor needs low latency, so a small model by default
`ENABLE_SIMULATORS`	true in dev; **forced false when** `NODE_ENV=production`	`/sim/*` and `/ops/*` routes

9. Test plan (maps to `test/`)

Area	Key cases
Binding	Happy path for both slots; SIMs of different owners allowed (ADR-004); airplane mode; absent SIM; 4th attempt blocked; expired token; same token from two numbers; token relayed from another phone (ADR-010); number rebound on another device deregisters the old profile
SIM integrity	Remove SIM 2 → only profile 2 suspended; re-insert → active with cooling-off; different SIM in slot 2 → deregistered
Accounts	Quota shared across profiles (25/day); balances and names not stored
Payments	Pay from each profile; pay from SIM 2's account with SIM 2 absent → rejected; profile signature from the wrong profile → rejected; idempotent replay returns the same txn without a second debit; conflicting replay; PENDING → status-check pacing → DEEMED_PENDING
Limits / risk	Cooling-off cap; daily limit; device velocity across profiles; new-payee confirm
Advisor	Ineligible reasons; deterministic ranking; a Claude response with a foreign ref or invented number falls back to rules; timeout falls back
Security	Bad signature, replayed nonce, tampered SIM-state header, PIN never in logs or DB, audit chain detects tampering

Appendix A: API specification (v1)

Base URL: `http://127.0.0.1:3000` (prototype). JSON bodies only. Money is in **paise** (integers).

Conventions

- **Authentication.** Every `/v1/*` call except `POST /v1/installations` needs the signed headers from LLD §3.2: `X-Install-Id`, `X-Timestamp`, `X-Nonce`, `X-Sim-State`, `X-Signature`. Payment and profile removal also need `X-Profile-Signature`.
- **SIM state header.** `X-Sim-State` is base64url JSON `{"airplane": false, "slots": [{"slot": 1, "state": "READY", "fp": "<64 hex>", "carrier": "404-10"}, {"slot": 2, "state": "ABSENT"}]}`. It is part of the signed string.
- **SIM enforcement.** The SIM integrity check (LLD §4.3) runs **before** every signed handler, and its outcome is returned in `simCheck` on every response.
- **Idempotency.** `POST /v1/payments` requires an `Idempotency-Key` header (UUID).
- **Errors.** Every error looks like `{"error": {"code": "SIM_ABSENT", "message": "..."}}`. Codes and HTTP statuses are in LLD §7.

Installations

`POST /v1/installations` (unsigned)

Registers the app install and its device public key.

```
// request
{ "publicKeyJwk": {"kty":"EC","crv":"P-256","x":"…","y":"…"}, "deviceModel": "Pixel 8",
"integrityToken": "…", "consentSimRead": true }
// 201
{ "installationId": "ins_…", "simSalt": "hex32" }
```

`consentSimRead` must be `true` (DPDP purpose: device binding and SIM integrity), otherwise `VALIDATION`.

`POST /v1/sim-state`

A heartbeat sent when the phone reports a SIM change. There is no body; it uses `X-Sim-State`.

```
// 200
{ "simCheck": [ {"profileId":"prf_…","slot":2,"action":"SUSPEND","reason":"SIM_ABSENT"} ] }
```

Bindings

`POST /v1/bindings`

```
// request
{ "slot": 2, "profilePublicKeyJwk": { … } }
// 201
{ "bindingId": "bnd_…", "token": "40-char one-time token", "vmn": "+919000000007", "expiresAt":
"2026-10-07T17:00:45Z" }
```

Errors: `AIRPLANE_MODE`, `SIM_ABSENT`, `SLOT_ALREADY_BOUND`, `PROFILE_CAP`, `BINDING_LIMIT`.

The app must then send `token` as an SMS **from that slot** to `vmn`. In the prototype this is done with `POST /sim/telecom/sms`.

`GET /v1/bindings/{id}`

```
{ "status": "PENDING" | "VERIFIED" | "FAILED" | "EXPIRED",
  "failReason": null | "TOKEN_EXPIRED" | "TOKEN_MULTI_SOURCE" | "VMN_MISMATCH" |
"SIM_NUMBER_MISMATCH" | "NUMBER_ALREADY_BOUND_ON_THIS_DEVICE",
  "profile": { "id": "prf_…", "slot": 2, "msisdnMasked": "70XXXXXX02", "vpa":
"dp8w2z0c7hn.2@duopay", "state": "BOUND" } }
```

Profiles and accounts

`GET /v1/profiles`

Lists every live profile on the installation, with its accounts.

```
{ "profiles": [ { "id": "prf_…", "slot": 1, "msisdnMasked": "98XXXXXX01", "vpa": "…", "state":
"ACTIVE",
    "coolingOffUntil": "…", "accounts": [ { "id": "acc_…", "bank": "BNKA", "bankName": "Bank A",
"maskedNumber": "XXXXXX1234", "type": "SAVINGS" } ] } ],
  "simCheck": [] }
```

`GET /v1/banks`

```
{ "banks": [ { "code": "BNKA", "name": "Bank A", "status": "UP" } ] }
```

`POST /v1/profiles/{id}/accounts/discover`

Runs `ReqListAccount` for **this profile's** number at one bank and links what it finds.

```
// request
{ "bankCode": "BNKC" }
// 200
{ "accounts": [ … ], "profileState": "ACTIVE", "quota": { "used": 3, "limit": 25 } }
```

Errors: `PROFILE_NOT_ACTIVE` (profile suspended or deregistered), `QUOTA_EXCEEDED`, `BANK_DOWN`.

`DELETE /v1/profiles/{id}`

Needs `X-Profile-Signature` for that profile. Returns `204`.

Payments

`POST /v1/payments/suggest`

```
// request
{ "payeeVpa": "chaiwala@psp", "amountPaise": 25000 }
// 200
{ "payee": { "vpa": "chaiwala@psp", "name": "Ravi Tea Stall" },
  "source": "claude" | "rules",
  "reason": "Bank C: you usually pay this payee from it",
  "eligible": [ { "accountId": "acc_…", "profileId": "prf_…", "slot": 2, "bankName": "Bank C",
"maskedNumber": "XXXXXX5678",
                  "score": 0.82, "headroom": { "amountLeftPaise": 9975000, "countLeft": 19,
"coolingLeftPaise": null } } ],
  "ineligible": [ { "accountId": "acc_…", "slot": 1, "bankName": "Bank A", "reason": "SIM_ABSENT"
} ],
  "simCheck": [] }
```

`POST /v1/payments`

Headers: `Idempotency-Key`, `X-Profile-Signature` (signed by the profile that owns `accountId`).

```
// request
{ "accountId": "acc_…", "payeeVpa": "chaiwala@psp", "amountPaise": 25000,
  "credBlock": "base64 from NPCI CL", "confirm": false, "suggestedRank": 1, "note": "tea" }
// 201 (or 200 when an idempotent replay returns the existing transaction)
{ "transaction": { "id": "txn_…", "status": "SUCCESS" | "FAILURE" | "PENDING", "rrn": "…",
"failureCode": null,
                   "amountPaise": 25000, "payeeVpa": "…", "fromSlot": 2, "bankName": "Bank C" } }
```

Errors: `PROFILE_NOT_ACTIVE`, `INVALID_SIGNATURE`, `LIMIT_EXCEEDED`, `COOLING_OFF_LIMIT`, `CONFIRM_REQUIRED` (with `rules`), `RISK_BLOCKED`, `PAYEE_INVALID`, `BANK_DOWN`, `IDEMPOTENCY_CONFLICT`.

A wrong PIN is **not** an HTTP error. It is a `FAILURE` transaction with `failureCode: "INCORRECT_PIN"`, as in UPI.

`GET /v1/payments?limit=20`

```
{ "transactions": [ … ] }
```

`POST /v1/payments/{id}/check-status`

```
{ "transaction": { … }, "checksUsed": 2, "nextAllowedAt": "…" }
```

Errors: `STATUS_CHECK_TOO_SOON`, `STATUS_CHECK_LIMIT`, `VALIDATION` (transaction not PENDING).

Simulator and ops endpoints (prototype only; disabled when `NODE_ENV=production`)

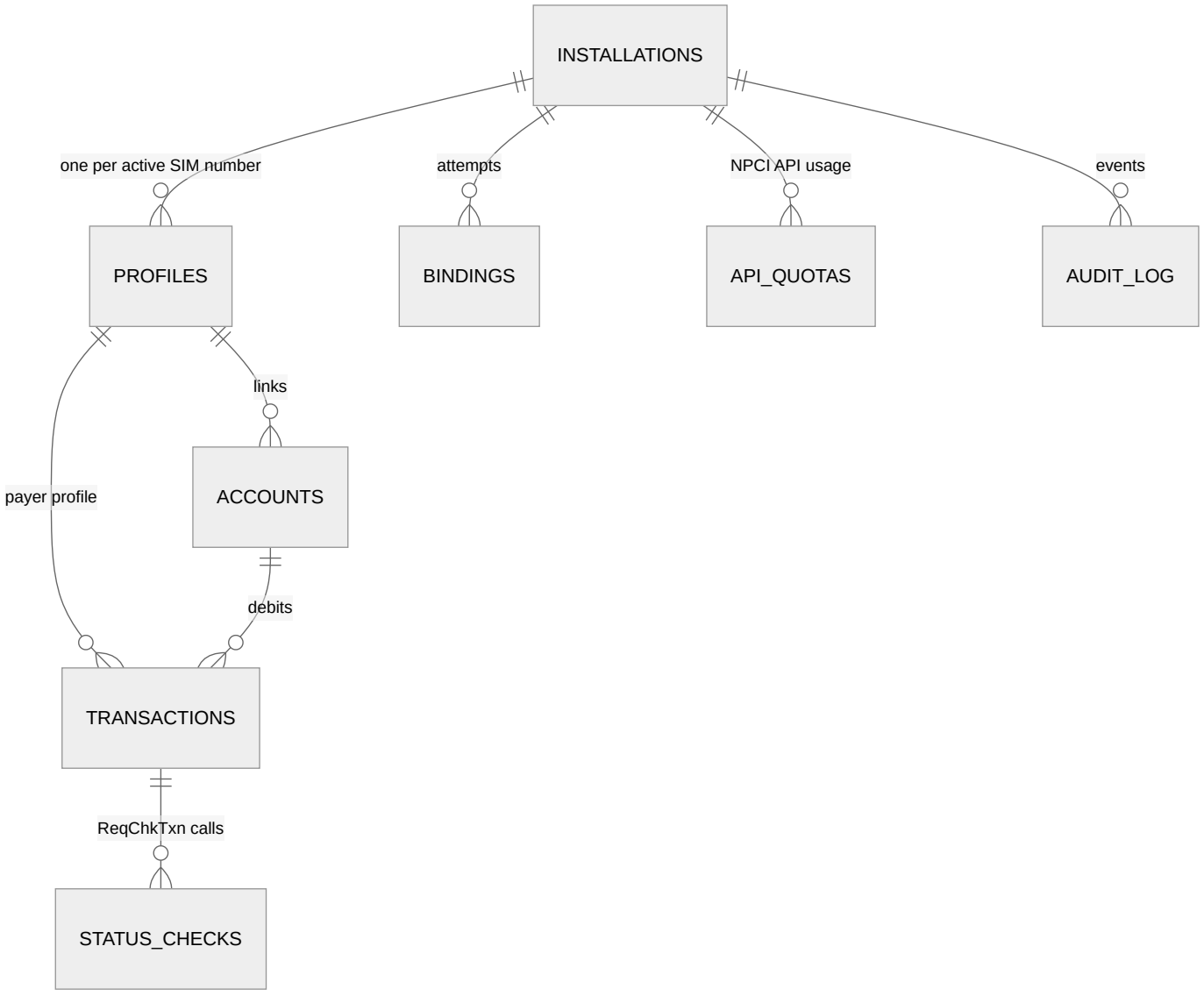
Method & path	Purpose
`GET /sim/world`	Fictional SIM cards (ICCID → number), banks, which number has which accounts, demo PINs
`POST /sim/telecom/sms` `{iccid, to, text}`	Send an SMS from a SIM. The telecom adds the originating number and forwards it to the PSP
`GET /sim/npci/keys?bank=BNKC`	Issuer public key (SPKI, base64) for the NPCI CL
`POST /sim/banks/{code}/mode` `{mode: "UP"	"DOWN"
`GET /ops/audit`	Audit ledger plus chain verification result

Method & path	Purpose
`GET /ops/profiles`	All profiles (masked) for the ops console

Appendix B: Database design

The prototype uses SQLite through the built-in `node:sqlite`. The DDL is written to port directly to PostgreSQL for production (HLD §13). The source of truth is [`src/db/schema.sql`](Appendix C).

Entity relationships



Tables

Table	Purpose	Key constraints
`installations`	One app install on one device: device public key, SIM salt, consent	PK `id`
`profiles`	**One per SIM number**: slot, encrypted number, bound SIM fingerprint, profile key, VPA, state	Partial unique indexes on live rows: one per `msisdn_hash` (system-wide), one per `vpa`, one per `(installation_id, slot)`

Table	Purpose	Key constraints
`bindings`	Device-binding attempts; only the token hash is stored; slot carrier for the ADR-010 check	`token_hash` unique; index for the 3-per-24h check
`accounts`	Linked bank accounts (masked), owned by exactly one profile	Unique `(profile_id, bank_code, account_ref)`
`transactions`	Payments; the id is the UPI transaction id	Unique `(installation_id, idempotency_key)`; `amount_paise > 0`
`status_checks`	Every `ReqChkTxn`, to enforce NPCI pacing	FK to transaction
`api_quotas`	Per-installation, per-day counters for NPCI-limited APIs	PK `(installation_id, api, day)`
`request_nonces`	Replay protection for signed requests	PK `nonce`; purged after 10 minutes
`audit_log`	Append-only, hash-chained event ledger	`seq` autoincrement; `hash` chains to `prev_hash`

What is deliberately *not* stored

Data	Why
Account balances	NPCI OC-95: only the bank may retain balance information
Account-holder names	Not needed (ADR-004 removed the same-owner check); data minimisation under DPDP
UPI PIN or the encrypted credential block	Captured by the NPCI CL; passed through once and never persisted or logged
Binding SMS token	Only its SHA-256 is stored
Raw ICCID	Only a salted fingerprint computed on the device
Full mobile number in clear	AES-256-GCM ciphertext + keyed hash + masked display form

Concurrency and integrity

- Each request's writes, including the nonce insert and the audit append, run in **one transaction** (`BEGIN IMMEDIATE` in SQLite; `SERIALIZABLE` or row locks in PostgreSQL).
- Limit checks read the day's SUCCESS + PENDING totals inside the same transaction that inserts the new `INITIATED` row. Two parallel payments therefore can't both pass a limit with headroom for only one.
- The partial unique indexes make "one live profile per number" a database guarantee, not just application logic.

Retention (configurable)

Data	Default retention
`transactions`, `status_checks`, `audit_log`	10 years (regulatory record)
`bindings`	180 days
`request_nonces`	10 minutes
`api_quotas`	30 days
Deregistered `profiles`	Number ciphertext erased after 180 days; hash kept for fraud linkage

Production notes

- PostgreSQL, partitioned by month on `transactions` and `audit_log`, with India-region primary and DR (RBI data localisation).
- Move `api_quotas`, `request_nonces` and idempotency lookups to Redis with TTLs.
- `DATA_KEY` in a KMS/HSM with envelope encryption and yearly rotation (store a `key_version` column with each ciphertext).

Appendix C: Full DDL

```
-- DuoPay schema (SQLite for the prototype; PostgreSQL-compatible types and constraints).
-- Money is stored in paise (INTEGER). Timestamps are ISO-8601 UTC strings.
-- Personal data: MSISDN is stored encrypted (AES-256-GCM) plus a keyed hash for lookups.
-- Balances are never stored (NPCI OC-95).

CREATE TABLE IF NOT EXISTS installations (
  id                TEXT PRIMARY KEY,                -- ins_xxx
  public_key_jwk    TEXT NOT NULL,                   -- ECDSA P-256 public key (device keystore)
  sim_salt          TEXT NOT NULL,                   -- per-install salt the app uses for SIM
fingerprints
  device_model      TEXT,
  integrity_verdict TEXT NOT NULL DEFAULT 'UNVERIFIED', -- MEETS_DEVICE_INTEGRITY | UNVERIFIED |
FAILED
  consent_sim_read  INTEGER NOT NULL DEFAULT 0,      -- DPDP consent flag
  created_at        TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS profiles (
  id               TEXT PRIMARY KEY,                 -- prf_xxx
  installation_id  TEXT NOT NULL REFERENCES installations(id),
  slot             INTEGER NOT NULL CHECK (slot BETWEEN 1 AND 4),
  msisdn_enc       TEXT NOT NULL,                    -- AES-256-GCM(iv:tag:ciphertext)
  msisdn_hash      TEXT NOT NULL,                    -- HMAC-SHA256(data key, msisdn)
  msisdn_masked    TEXT NOT NULL,                    -- 98XXXXXX01
  sim_fp           TEXT NOT NULL,                    -- fingerprint bound at binding time
  public_key_jwk   TEXT NOT NULL,                    -- per-profile signing key
  vpa              TEXT NOT NULL UNIQUE,             -- random; never reissued
  state            TEXT NOT NULL CHECK (state IN ('BOUND','ACTIVE','SUSPENDED','DEREGISTERED')),
  state_reason     TEXT,
  activated_at     TEXT,                             -- start of cooling-off window
  suspended_at     TEXT,
  created_at       TEXT NOT NULL,
  updated_at       TEXT NOT NULL
);
-- One live profile per SIM number across the whole system (a number is bound to one device at a
time)
CREATE UNIQUE INDEX IF NOT EXISTS ux_profiles_live_msisdn ON profiles(msisdn_hash) WHERE state <>
'DEREGISTERED';
-- One live profile per slot per installation
CREATE UNIQUE INDEX IF NOT EXISTS ux_profiles_live_slot ON profiles(installation_id, slot) WHERE
state <> 'DEREGISTERED';

CREATE TABLE IF NOT EXISTS bindings (
  id               TEXT PRIMARY KEY,                 -- bnd_xxx
  installation_id  TEXT NOT NULL REFERENCES installations(id),
  slot             INTEGER NOT NULL,
  sim_fp           TEXT NOT NULL,
  carrier          TEXT NOT NULL,                    -- MCC-MNC the device reported for this slot
(ADR-010)
  token_hash       TEXT NOT NULL UNIQUE,             -- SHA-256 of the SMS token (token itself
never stored)
  vmn              TEXT NOT NULL,
  profile_key_jwk  TEXT NOT NULL,
  status           TEXT NOT NULL CHECK (status IN ('PENDING','VERIFIED','FAILED','EXPIRED')),
  fail_reason      TEXT,
  received_msisdn_hash TEXT,
  profile_id       TEXT REFERENCES profiles(id),
  expires_at       TEXT NOT NULL,
  created_at       TEXT NOT NULL
);
CREATE INDEX IF NOT EXISTS ix_bindings_attempts ON bindings(installation_id, slot, created_at);
CREATE INDEX IF NOT EXISTS ix_bindings_number ON bindings(received_msisdn_hash, created_at);

CREATE TABLE IF NOT EXISTS accounts (
  id               TEXT PRIMARY KEY,                 -- acc_xxx
  profile_id       TEXT NOT NULL REFERENCES profiles(id),
  bank_code        TEXT NOT NULL,
  account_ref      TEXT NOT NULL,                    -- opaque reference returned by the bank
  masked_number    TEXT NOT NULL,                    -- XXXXXX1234
  ifsc             TEXT NOT NULL,
  account_type     TEXT NOT NULL,                    -- SAVINGS | CURRENT | OD
```

```
  created_at       TEXT NOT NULL,
  UNIQUE (profile_id, bank_code, account_ref)
);

CREATE TABLE IF NOT EXISTS transactions (
  id               TEXT PRIMARY KEY,                 -- txn_xxx (also the UPI txn id sent to the
PSP)
  installation_id  TEXT NOT NULL REFERENCES installations(id),
  profile_id       TEXT NOT NULL REFERENCES profiles(id),
  account_id       TEXT NOT NULL REFERENCES accounts(id),
  idempotency_key  TEXT NOT NULL,
  request_hash     TEXT NOT NULL,                    -- detects reuse of a key with a different
body
  payee_vpa        TEXT NOT NULL,
  payee_name       TEXT,
  amount_paise     INTEGER NOT NULL CHECK (amount_paise > 0),
  status           TEXT NOT NULL CHECK (status IN
('INITIATED','SUCCESS','FAILURE','PENDING','DEEMED_PENDING')),
  rrn              TEXT,                             -- bank reference number
  failure_code     TEXT,
  suggested_rank   INTEGER,                          -- position of the chosen account in the
suggestion (null = no suggestion)
  created_at       TEXT NOT NULL,
  updated_at       TEXT NOT NULL,
  UNIQUE (installation_id, idempotency_key)
);
CREATE INDEX IF NOT EXISTS ix_txn_account_day ON transactions(account_id, created_at);
CREATE INDEX IF NOT EXISTS ix_txn_install_time ON transactions(installation_id, created_at);

CREATE TABLE IF NOT EXISTS status_checks (
  id               INTEGER PRIMARY KEY AUTOINCREMENT,
  transaction_id   TEXT NOT NULL REFERENCES transactions(id),
  checked_at       TEXT NOT NULL,
  result           TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS api_quotas (
  installation_id  TEXT NOT NULL,
  api              TEXT NOT NULL,                    -- LIST_ACCOUNT | BAL_ENQ
  day              TEXT NOT NULL,                    -- YYYY-MM-DD (IST)
  count            INTEGER NOT NULL DEFAULT 0,
  PRIMARY KEY (installation_id, api, day)
);

CREATE TABLE IF NOT EXISTS request_nonces (
  nonce            TEXT PRIMARY KEY,
  installation_id  TEXT NOT NULL,
  seen_at          TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS audit_log (
  seq              INTEGER PRIMARY KEY AUTOINCREMENT,
  at               TEXT NOT NULL,
  installation_id  TEXT,
  profile_id       TEXT,
  event            TEXT NOT NULL,                    -- e.g. PROFILE_SUSPENDED, PAYMENT_SUCCESS
  data             TEXT NOT NULL,                    -- JSON, no PII beyond masked values
  prev_hash        TEXT NOT NULL,
  hash             TEXT NOT NULL                     -- SHA-256(prev_hash || canonical row)
);
```